Комбинаторика как раздел математики

В современном мире ***комбинаторные алгоритмы*** позволяют реализовывать сложнейшие вычисления: например, они используются при расшифровке ДНК для определения наследственной предрасположенности к заболеваниям, при анализе динамики стоимости ценных бумаг, для разработки методов шифрования.

> **Комбинаторика** – раздел математики, в котором изучаются различные соединения (комбинации) элементов конечных множеств.

Похожее место занимает и ***теория вероятностей***. Когда-то она появилась как инструмент для прогнозирования успеха в казино, однако сейчас помогает, например, генерировать тексты, отделять спам от обычных писем, выводить новые сорта растений и изучать потоки космических частиц.

Правило сложения и умножения

Многие комбинаторные задачи могут быть решены с помощью двух правил – правила умножения и правила сложения.

> **Правило суммы**. Если некоторый объект A может быть выбран из совокупности объектов m способами, а другой объект B может быть выбран n способами, то выбрать либо A, либо B можно $m + n$ способами.

> **Правило произведения**. Если объект A можно выбрать из совокупности объектов m способами и после каждого такого выбора объект B можно выбрать n способами, то пара объектов (A, B) в указанном порядке может быть выбрана $m \cdot n$ способами.

Эти правила распространяются на любое конечное число объектов.

Пример 1. Сколькими способами пять человек могут встать в очередь?

Пример 2. Сколько однозначных и сколько двузначных чисел можно составить из цифр от 0 до 9 (повторяя и не повторяя цифры в числе)?

Основные виды комбинаторных объектов

Существуют две схемы выбора k элементов из заданного множества: ***без повторения***, когда выбранные элементы не возвращаются в исходное множество, и ***с повторением***, когда выбор осуществляется поэлементно с обязательным возвращением отобранного элемента на каждом шаге.

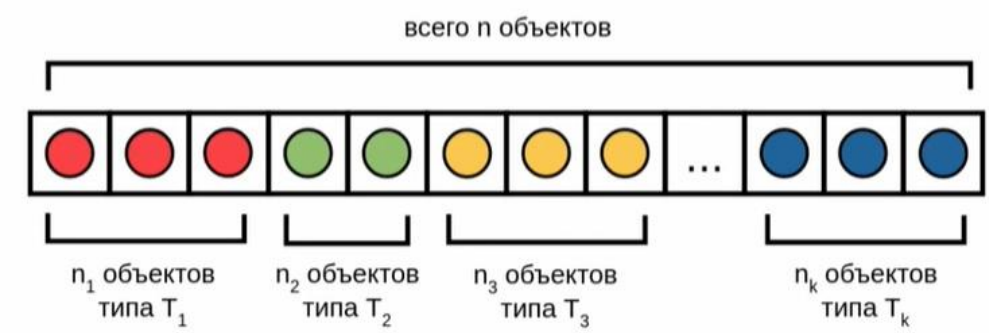
Перестановки

> **Перестановкой** из n элементов называется любая выборка n элементов без повторений и с учетом порядка следования элементов.

$$P_n = n!$$

Перестановка с повторением – это выборка n элементов из n элементов, среди которых n_1 элементов первого типа, n_2 элементов второго типа, ..., n_k элементов k типа, причем $n_1 + n_2 + \ldots + n_k = n$. Число всех таких перестановок с повторениями:

$$P_n(n_1, n_2, \ldots, n_k) = \frac{n!}{n_1! \cdot n_2! \cdot \ldots \cdot n_k!}$$



Размещения

Размещение без повторений – это выборка из n элементов k элементов без повторений и с учетом порядка следования элементов.

$$A_n^k = \frac{n!}{(n-k)!}$$

Размещение с повторением – это выборка из n элементов k с повторениями и с учетом порядка следования элементов.

$$\overline{A_n^k} = n^k$$

Сочетания

Сочетание без повторения – это выборка из n элементов k элементов без повторения и без учета порядка следования элементов.

$$C_n^k = \frac{n!}{k!(n-k)!}$$

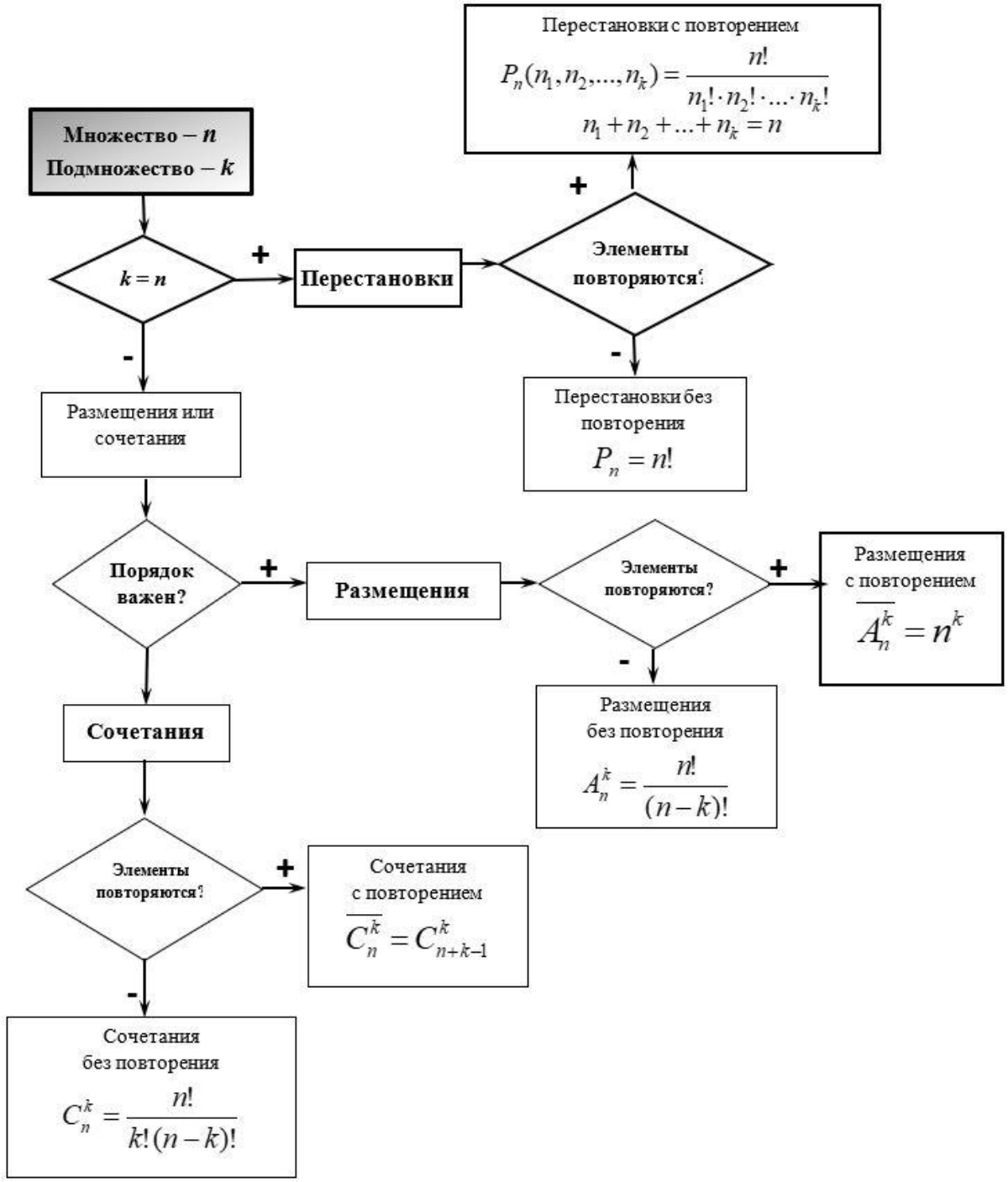
Сочетание с повторением - это выборка из n элементов k с повторением и без учета порядка следования элементов (причем любой объект может входить в соединение любое число раз, но не более k раз).

$$\overline{C_n^k} = C_{n+k-1}^k$$

Итоговая сводка формул приведена в следующей таблице

Тип выборки	Перестановки	Размещения	Сочетания
Без повторения	$P_n = n!$	$A_n^k = \frac{n!}{(n-k)!}$	$C_n^k = \frac{n!}{k!(n-k)!}$
С повторением	$P_n(n_1, n_2, \ldots, n_k) = \frac{n!}{n_1! \cdot n_2! \cdot \ldots \cdot n_k!}$	$\overline{A_n^k} = n^k$	$\overline{C_n^k} = C_{n+k-1}^k$

Алгоритм решения комбинаторной задачи



Свойство числа сочетаний

Приведем свойства числа сочетаний, которые могут быть полезными при решении комбинаторных задач.

Для чисел C_n^k (они называются биномиальными коэффициентами) справедливы следующие тождества:

1. Правило симметрии: $C_n^k = C_{n-k}^k$
2. Правило Паскаля: $C_n^k = C_{n-1}^k + C_{n-1}^{k-1}$
3. $C_n^0 + C_n^1 + \ldots + C_n^n = 2^n$
4. $C_n^0 = C_n^n = 1$

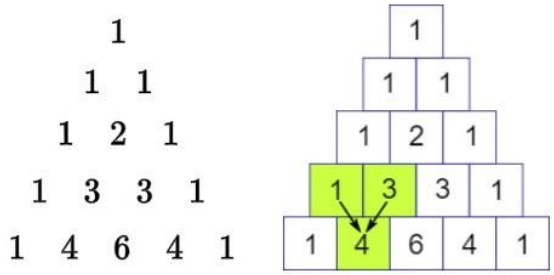
Треугольник Паксаля

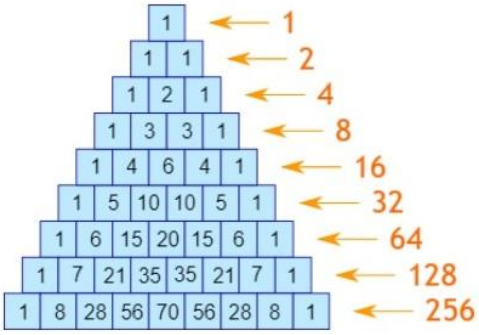
$$(x+y)^0 = 1$$

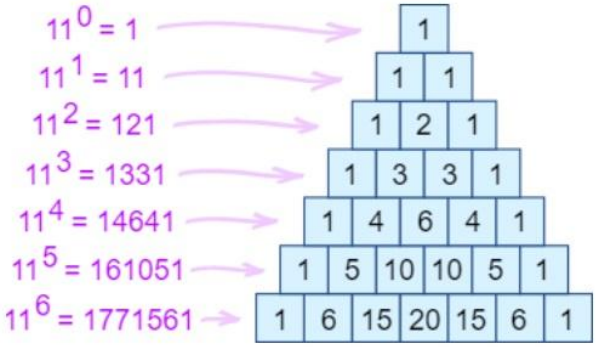
$$(x+y)^1 = 1 \cdot x + 1 \cdot y$$

$$(x+y)^2 = 1 \cdot x^2 + 2 \cdot xy + 1 \cdot y^2$$

$$(x+y)^3 = 1 \cdot x^3 + 3 \cdot x^2 y + 3 \cdot xy^2 + 1 \cdot y^3$$







Комбинаторика в Python

Библиотека `itertools`

Стандартная библиотека python `itertools` позволяет **генерировать комбинации**. Она содержит ряд функций, позволяющих увидеть сами комбинаторные объекты:

- `product` перебирает все с повторениями, длина указывается в параметре `repeat` .
- `permutations` генерирует все перестановки для итерируемого объекта, длина указывается в параметре `r` .
- `combinations` (`iterable` , [`r`]) - комбинации длиной `r` из `iterable` без повторяющихся элементов.

Перестановки

Ввод [4]:

```
# Перестановки без повторений
from itertools import permutations

setItem = '5678'
k = len(setItem)

a = list(permutations(setItem, r=k))

print('Всего комбинаций: {:d}.'.format(len(a)))
for item in a:
    print(item)
```

```
Всего комбинаций: 24.
('5', '6', '7', '8')
('5', '6', '8', '7')
('5', '7', '6', '8')
('5', '7', '8', '6')
('5', '8', '6', '7')
('5', '8', '7', '6')
('6', '5', '7', '8')
('6', '5', '8', '7')
('6', '7', '5', '8')
('6', '7', '8', '5')
('6', '8', '5', '7')
('6', '8', '7', '5')
('7', '5', '6', '8')
('7', '5', '8', '6')
('7', '6', '5', '8')
('7', '6', '8', '5')
('7', '8', '5', '6')
('7', '8', '6', '5')
('8', '5', '6', '7')
('8', '5', '7', '6')
('8', '6', '5', '7')
('8', '6', '7', '5')
('8', '7', '5', '6')
('8', '7', '6', '5')
```

Ввод [5]:

```
# Перестановки с повторениями
from itertools import product

setItem = 'AB'

a = set(product(setItem, repeat=4))

print('Всего комбинаций: {:d}.'.format(len(a)))
for item in a:
    print(item)
```

```
Всего комбинаций: 16.
('A', 'A', 'B', 'A')
('A', 'B', 'A', 'B')
('A', 'A', 'A', 'B')
('B', 'B', 'A', 'A')
('B', 'B', 'B', 'B')
('B', 'A', 'B', 'A')
('B', 'B', 'A', 'B')
('B', 'A', 'A', 'A')
('A', 'A', 'B', 'B')
('A', 'B', 'B', 'A')
('A', 'B', 'A', 'A')
('A', 'A', 'A', 'A')
('B', 'A', 'B', 'B')
('B', 'B', 'B', 'A')
('B', 'A', 'A', 'B')
('A', 'B', 'B', 'B')
```

Размещения

Ввод [6]:

```
# Размещения без повторений
from itertools import permutations

setItem = 'ABCD'
k = 2

a = list(permutations(setItem, r=k))
print('Всего комбинаций: {:d}.'.format(len(a)))
for item in a:
    print(item)
```

```
Всего комбинаций: 12.
('A', 'B')
('A', 'C')
('A', 'D')
('B', 'A')
('B', 'C')
('B', 'D')
('C', 'A')
('C', 'B')
('C', 'D')
('D', 'A')
('D', 'B')
('D', 'C')
```

Ввод [7]:

```
# Размещения с повторениями
from itertools import product

setItem = 'AB'

a = set(product(setItem, repeat=2))
print('Всего комбинаций: {:d}.'.format(len(a)))
for item in a:
    print(item)
```

```
Всего комбинаций: 4.
('B', 'A')
('A', 'B')
('A', 'A')
('B', 'B')
```

Сочетания

Ввод [11]:

```
# Сочетания без повторений
from itertools import combinations

setItem = 'ABCD'
k = 3

a = set(combinations(setItem, r=k))

print('Всего комбинаций: {:d}.'.format(len(a)))

for item in a:
    print(item)
```

```
Всего комбинаций: 4.
('B', 'C', 'D')
('A', 'B', 'C')
('A', 'B', 'D')
('A', 'C', 'D')
```

Ввод [10]:

```
# Сочетания с повторениями
from itertools import combinations_with_replacement

setItem = 'ABCD'
k = 3

a = list(combinations_with_replacement(setItem, r=k))

print('Всего комбинаций: {:d}.'.format(len(a)))

for item in a:
    print(item)
```

```
Всего комбинаций: 20.
('A', 'A', 'A')
('A', 'A', 'B')
('A', 'A', 'C')
('A', 'A', 'D')
('A', 'B', 'B')
('A', 'B', 'C')
('A', 'B', 'D')
('A', 'C', 'C')
('A', 'C', 'D')
('A', 'D', 'D')
('B', 'B', 'B')
('B', 'B', 'C')
('B', 'B', 'D')
('B', 'C', 'C')
('B', 'C', 'D')
('B', 'D', 'D')
('C', 'C', 'C')
('C', 'C', 'D')
('C', 'D', 'D')
('D', 'D', 'D')
```

Математические функции в модуле scipy.special

Для импорта библиотеки используется сторка вида:

```
import scipy.special as sc
```

Для подсчета количества комбинаций используются:

- `sc.factorial(n)` - подсчет числа перестановок из $n!$ элементов;
- `sc.perm(n, k)` - число размещений A_n^k (из n по k);
- `sc.comb(n, k)` - число сочетаний C_n^k (из n по k);
- `sc.binom(n, k)` - биномиальные коэффициенты C_n^k.

Пример. Сколькими способами можно расставить на полке 5 книг?

Ввод [9]:

```
import scipy.special as sc

n = 5
print(sc.factorial(n))
```

```
120.0
```

Пример. Сколькими способами можно выбрать в группе из 25 человек старосту и его заместителя?

Ввод [10]:
```
import scipy.special as sc

n = 25
k = 2
print(sc.perm(n, k))
```

```
600.0
```

Пример. Вычислить A_5^4, A_6^3, A_7^2.

Ввод [11]:
```
import scipy.special as sc

sc.perm([5, 6, 7],[4, 3, 2])
```

Out[11]:
```
array([120., 120.,  42.])
```

Пример. Сколькими способами можно выбрать в группе из 25 человек двух дежурных?

Ввод [12]:
```
import scipy.special as sc

sc.comb(25, 2)
```

Out[12]:
```
np.float64(300.0)
```

Пример. Вычисление последовательности биномиальных коэффициентов C_3^0, C_3^1, C_3^2, C_3^3.

Ввод [13]:
```
import scipy.special as sc

print(sc.comb([3] * 4, [0, 1, 2, 3]))
print(sc.binom([3, 3, 3, 3], range(4)))
```

```
[1. 3. 3. 1.]
[1. 3. 3. 1.]
```

Ввод [22]:
```
# Треугольник Паскаля
import scipy.special as sc

lim = 10

for n in range(lim):
    print(*map(int, sc.comb([n-1] * n, range(n))))
```

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
```

Ввод [10]:
```
import scipy.special as sc

print(sc.binom([6] * 7, [0, 1, 2, 3, 4, 5,6]))
```

```
[ 1.  6. 15. 20. 15.  6.  1.]
```

Практика. Решение задач

Задача (автомобильные номера)

Подсчитать количество возможных автомобильных номеров в России.



Справка. Структура номера: Российский автомобильный номер состоит из трех букв и трех цифр, а также кода региона. Количество возможных комбинаций ограничено используемыми буквами и цифрами.только те буквы кириллицы, которые имеют аналоги в латинском алфавите. Это буквы A, B, E, K, M, H, O, P, C, T, У, X. Таким образом, для каждой позиции буквы доступно 12 вариантов.

Ввод [23]:
```
# по правилу произведения
12 * (10 * 10 * 10 - 1) * 12 * 12
```

Out[23]:
```
1726272
```

Задача (интеллектуальная мартышка)

В зоопарке мартышке вместо игрушек подкидывают карточки с буквами. А мартышка от безделия считает сколько всего комбинаций (включая неосмысленные) она может составить из всех карточек, меняя порядок их следования в комбинации. Только нет у нее правильных ответов и она не может проверить количество комбинаций, которые она получила, когда ей дали:

- карточки со словом *ГИПЕРБОЛА*;
- карточки со словом *КОЛОБОК*.

Ввод [31]:
```
# слово ГИПЕРБОЛА
from itertools import permutations
from scipy.special import perm

alf = 'ГИПЕРБОЛА'
d = len(alf)

ls = list(permutations(alf, r=d))
print(len(ls))

print(perm(d, d))
```

```
362880
362880.0
```

Ввод [30]:

```
# слово КОЛОБОК
from itertools import permutations
from scipy.special import factorial

alf = 'КОЛОБОК'
d = len(alf)

ls = set(permutations(alf, r=d))
print(len(ls))

zn = 1
for c in set(alf):
    zn *= factorial(alf.count(c))

print(factorial(d)/zn)
```

```
420
420.0
```

Задача (выбор кандидатов)

В группе 20 человек. Необходимо выбрать троих. Сколько существует способов это сделать, если:

- выбраются староста, его заместитель и профорг;
- выбираются дежурные в школьной столовой.

Задача (соседка интеллектуальной мартышки)

Соседка интеллектуальной мартышки по "камере", решила затмить соим интеллектом боевую подругу. И, пока та переставляла карточки в слове "КОЛОБОК", забрала карточки со словом "ГИПЕРБОЛА" и стала считать количество комбинаций (включая неосмысленные), которые можно составить из этих карточек, используя любое их количество.

Ввод [38]:

```
from itertools import permutations
from scipy.special import perm

alf = 'ГИПЕРБОЛА'
d = len(alf)

ls = []
for ln in range(1, d + 1):
    ls.extend([''.join(el) for el in list(permutations(alf, ln))])
# print(*ls[::100])
print(len(ls))

cnt = 0
for ln in range(1, d + 1):
    cnt += perm(d, ln)
print(cnt)
```

```
986409
986409.0
```

Задача (комбинации в покере)

В колоде 52 карты, каждая из которых имеет одно из 13 значений $\{T, 2, 3, \ldots, 10, B, D, K\}$ и 4 мастей $\{\clubsuit, \spadesuit, \blacklozenge, \heartsuit\}$, Комбинация в покере состоит из 5 карт. Требуется найти количество способов выпадения следующих комбинаций:

1. 4 туза;
2. «четверка» (учитывая 4 тузов);
3. «одна пара» (только одна, три одинаковые по весу карты не дают «одну пару»).



При выборе думаем про вес карты и про масть.

Ввод [3]:
```
# событие A - выпало 4 туза
n_A = 1 * 48    # 1 - способов выбора 4 тузов,
                # 48 - способов выбора пятой карты
n_A
```

Out[3]: 48

Ввод [4]:
```
# событие B - выпала "четверка"
n_B = 13 * 1 * 48   # 13 - способов выбора веса карты,
                    # 1 - выбор мастей для четверки,
                    # 48 - способов выбора пятой карты (12 * 4)
n_B
```

Out[4]: 624

Ввод [2]:
```
from scipy.special import comb

# событие  C - выпала "одна пара"
n_C = 13 * comb(4, 2) * comb(12, 2) * 4 ** 3
      # 13 - способов выбора весов,
      # C(4, 2) - выбор масти для 2 карт из 4
      # C(12, 3) - выбор веса для трех карт (которые не в паре)
      # 4^3 - варианты выбора мастей
n_C
```

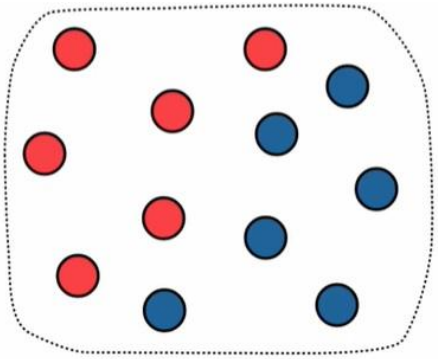
Out[2]: np.float64(329472.0)

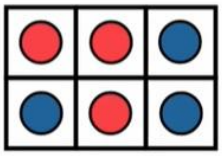
Задача (красные и синие шарики)

В коробке находится 6 красных и 6 синих шариков.Сколько существует способов вынуть:

1. 6 любых шариков;

2. 3 красных и 3 синих.





Комбинация

Ввод [17]:
```
# событие A - выбор шести шариков
# (можно пронумеровать и не учитывать порядок номеров)
from scipy.special import comb

n_A = comb(12, 6)
n_A
```

Out[17]: np.float64(924.0)

Ввод [19]:
```
# событие B - выбор 3 красных и 3 синих шариков
from scipy.special import comb

n_B = comb(6, 3) * comb(6, 3)
n_B
```

Out[19]: np.float64(400.0)

Задача (пермия "Оскар")

На премию "Оскар" в 5 номинациях рассматривают 10 кинофильмов. Сколько существует вариантов распределения призов, если по каждой номинации установлены:

- различные призы;
- одинаковые призы?

Задача (футбольные матчи)

В футбольном сезоне 38 игр. Дан футбольный клуб, который может выиграть, проиграть и сыграть вничью с равной вероятностью. Сколько существует комбинаций, соответствующих сезону с 20 победами, 11 ничьими и 7 поражениями для данного клуба?

Ввод [25]:
```
from scipy.special import factorial

n_A = factorial(38) / (factorial(20) * factorial(11) * factorial(7))
n_A
```

Out[25]: np.float64(1068586291412640.0)

Домашняя работа

Самостоятельно надо:

- разобрать решение задач §1.5, §1.6 (стр. 23) в учебнике Кремера